

TRUSTPAY

AI-Powered Fraud Detection & Asynchronous Event-Driven Banking Platform

A publication-grade engineering report presenting the 6-service containerized architecture, RabbitMQ asynchronous ML decoupling, Redis ZSET sliding-window rate limiting & caching, Boyce-Codd Normal Form (BCNF) relational ledger, ACID row-level locking, sub-2ms LightGBM fraud inference, and Groq Llama 3.3 GenAI forensic explanations.

6 Services

Dockerized Topology

BCNF + MQ

MySQL 8 • RabbitMQ • Redis

99.98%

ML Accuracy / 100% Prec

1.31 ms

Mean Inference Latency

PROJECT CONTRIBUTORS & STUDENT CREDENTIALS

Raghav Gupta

20243226

B.Tech (3rd Year) • CSE

Rishabh Srivastava

20243236

B.Tech (3rd Year) • CSE

Rishabh Singh

20243235

B.Tech (3rd Year) • CSE

Prince Keshari

20243218

B.Tech (3rd Year) • CSE

RabbitMQ Asynchronous ML Decoupling

Non-blocking `202_Accepted` gateway (<15ms) publishes to durable `fraud_evaluation_queue`; FastAPI `aio-pika` worker settles MySQL atomically.

Dual-AI Explainability & ACID Locks

Combines 14-feature LightGBM scoring with Groq Llama 3.3 70B rationales and row-locked `FOR UPDATE` stored procedures.

Redis 7 Rate Limiting & Caching

Sorted-Set `ZSET` sliding-window rate limiter (5 req / 10s → `HTTP 429`) and sub-2ms TTL caching for `/accounts/me` & `/auth/profile`.

6-Container Docker Compose & Benchmarks

Orchestrates all 6 services with healthchecks; benchmarked across 100k project tx (99.98%), ULB (284k tx), and IEEE-CIS (30k tx).

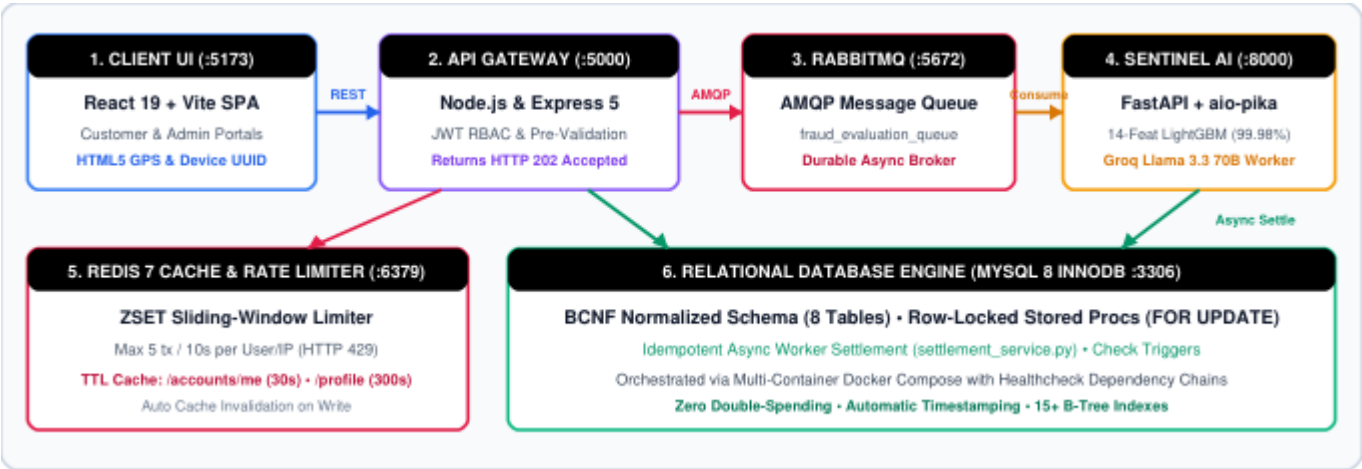
Data & Broker: MySQL 8 • RabbitMQ • Redis 7

Gateway & AI: Express 5 • FastAPI • LightGBM

Verification: Production Verified v2.0

1. Executive Summary & 6-Service Distributed Topology

TRUSTPAY is an enterprise financial technology platform engineered to process high-throughput banking transactions while executing asynchronous multi-tiered artificial intelligence risk analysis. The upgraded system architecture coordinates **six containerized services** orchestrated via Docker Compose: a React 19 SPA, an Express 5 API Gateway, a RabbitMQ AMQP Broker, a Redis 7 Cache & Sliding-Window Rate Limiter, a FastAPI Sentinel AI Worker (LightGBM + Groq Llama 3.3 70B), and a BCNF-normalized MySQL 8 relational ledger.



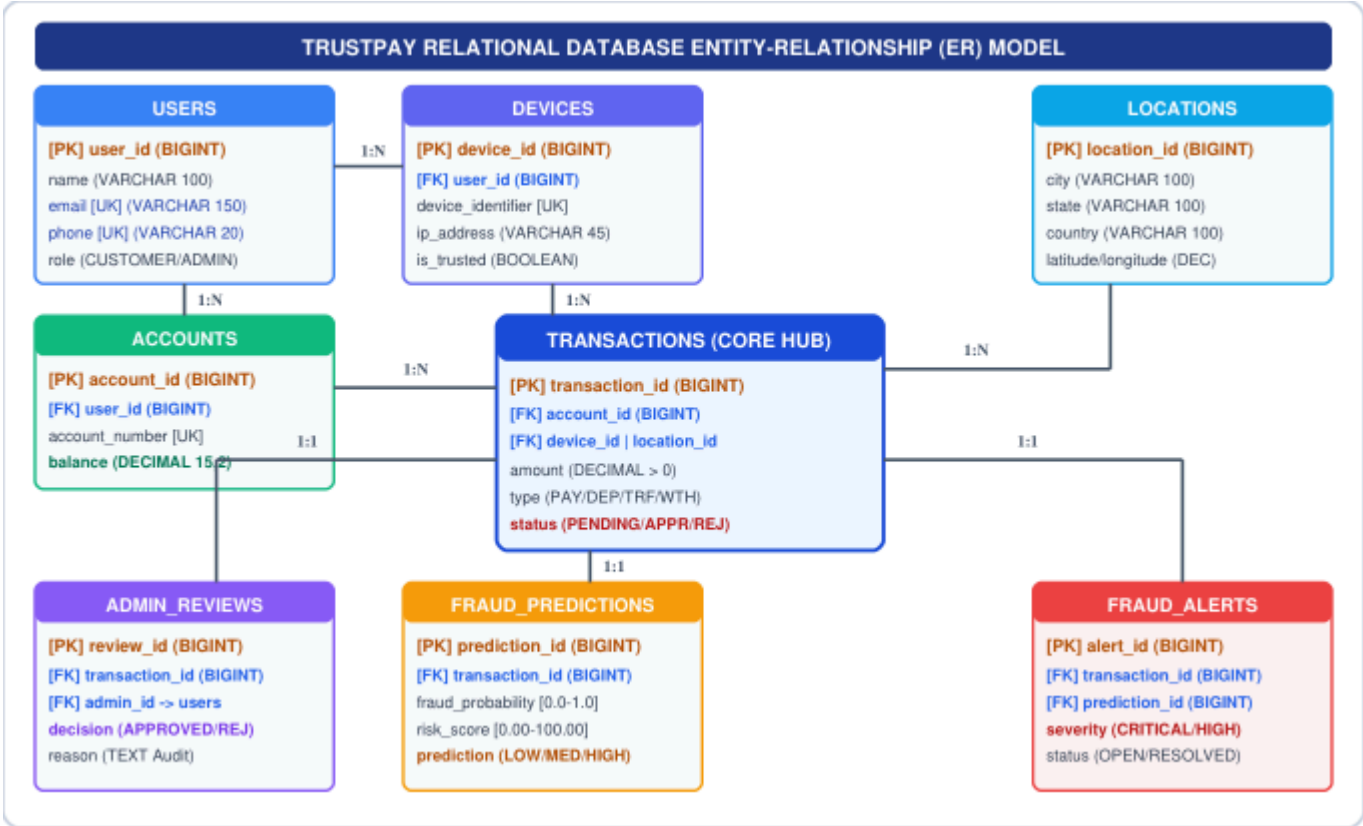
Service Tier	Core Technology	Operational Responsibility	Status
1. Client UI (:5173)	React 19, Vite, Router v7	Customer banking portal, GPS telemetry, and Admin Sentinel compliance cockpit	Active
2. API Gateway (:5000)	Node.js 20, Express 5, amqplib	Stateless JWT RBAC, balance pre-validation, non-blocking 202 Accepted producer	Active
3. RabbitMQ (:5672)	RabbitMQ 3 Management	Durable fraud_evaluation_queue decoupling ~1.43s ML/LLM evaluation from Node	Active
4. Redis 7 (:6379)	Redis 7 Alpine (AOF)	ZSET sliding-window rate limiter (5 req/10s) & TTL cache (/accounts/me, /profile)	Active
5. Sentinel AI (:8000)	FastAPI, aio-pika, LightGBM	Async queue consumer, 14-feature LightGBM model, Groq Llama 3.3 & DB settlement	Active
6. MySQL 8 DB (:3306)	MySQL 8.0 InnoDB	BCNF schema (8 tables), row-level locks (FOR UPDATE), triggers & stored procs	Active

1.1 Architectural Report Table of Contents (12 Pages)

Page 2: Executive Summary & 6-Service Topology Page 3: Relational Database (ER Model & BCNF) Page 4: Master Entity Catalog & Data Dictionary Page 5: Concurrency Control & Core API Gateway Page 6: RabbitMQ Async Queue, Redis & Docker Compose Page 7: Frontend Layer & 20-Point UI/UX Audit	Page 8: 7-Phase End-to-End Surveillance Pipeline Page 9: Sentinel AI Intelligence Architecture Page 10: Empirical ML Accuracy & Latency Benchmarks Page 11: Real-World Cross-Dataset Benchmarks (ULB & IEEE) Page 12: Cloud Deployment & Academic Defense Sign-Off
--	--

2. Normalized Relational Database Architecture

The TrustPay relational database engine operates on MySQL 8 with InnoDB tables enforcing strict foreign key constraints. The schema satisfies Boyce-Codd Normal Form (BCNF), eliminating redundancy, multi-valued dependencies, and transitive update anomalies across 8 distinct entities.



2.1 Relational Normalization Calculus (1NF to BCNF)

- 1NF (Atomic Attributes):** All table columns represent scalar, atomic values. Geolocation coordinates (lat/long) and client device fingerprints are decomposed into dedicated tables rather than serialized or delimited strings.
- 2NF (Full Functional Dependency):** All non-prime attributes are completely dependent on candidate keys. Composite natural keys were extracted into distinct entities with surrogate primary keys.
- 3NF (Transitive Dependency Removal):** Non-key attributes depend exclusively on primary keys ($X \rightarrow Y$ with no $Y \rightarrow Z$). Geolocation attributes were decoupled into `locations`, preventing city/country duplication within `transactions`.
- BCNF (Strict Determinant Keys):** For every functional dependency $X \rightarrow Y$, X is a superkey. There are no overlapping candidate keys or dependencies on non-candidate attributes.

Referential Integrity Strategy: Child operational records (`accounts`, `devices`, `fraud_predictions`, `fraud_alerts`) cascade on user deletion. Conversely, `transactions` utilizes `ON DELETE RESTRICT` on accounts and `ON DELETE SET NULL` on devices and locations, guaranteeing audit survivability.

2.2 Master Entity Catalog & Data Dictionary

The relational ledger comprises 8 Boyce-Codd Normal Form (BCNF) compliant entities enforcing strict referential integrity, automated check triggers, and index optimization.

Entity Name	Primary Key	Foreign Keys & Cascades	Key Constraints	Business Role
users	user_id (BIGINT)	None	UNIQUE(email), UNIQUE(phone), role IN ('CUSTOMER', 'ADMIN')	Customer & Admin master identity
accounts	account_id (BIGINT)	user_id -> users (CASCADE)	UNIQUE(account_number), balance >= 0	Bank account liquidity ledger
devices	device_id (BIGINT)	user_id -> users (CASCADE)	UNIQUE(user_id, device_identifier)	Hardware profiles & client trust
locations	location_id (BIGINT)	None	Decoupled GPS lat/long & city attributes	Spatial master directory
transactions	transaction_id (BIGINT)	account_id (RESTRICT) device_id (SET NULL) location_id (SET NULL)	CHECK(amount > 0), status IN ('PENDING', 'APPROVED', 'REJECTED')	Core immutable financial ledger
fraud_predictions	prediction_id (BIGINT)	transaction_id (CASCADE)	CHECK(fraud_probability BETWEEN 0 AND 1), CHECK(risk_score BETWEEN 0 AND 100)	Output of LightGBM ML inference
fraud_alerts	alert_id (BIGINT)	transaction_id (CASCADE) prediction_id (CASCADE)	severity IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'), status IN ('OPEN', 'RESOLVED')	High-risk security triage queue
admin_reviews	review_id (BIGINT)	transaction_id (RESTRICT) admin_id -> users (RESTRICT)	decision IN ('APPROVED', 'REJECTED')	Immutable compliance audit log

2.3 Active Database Triggers

- trg_transaction_before_insert** (BEFORE INSERT ON **transactions**): Enforces monetary integrity by intercepting transactions where `amount ≤ 0` and terminating execution with `SIGNAL SQLSTATE '45000'`.
- trg_fraud_alert_resolved** (BEFORE UPDATE ON **fraud_alerts**): Automates lifecycle auditing by setting `resolved_at = CURRENT_TIMESTAMP` whenever an alert transitions to `'RESOLVED'`.
- trg_fraud_alert_before_insert** (BEFORE INSERT ON **fraud_alerts**): Defensive fallback ensuring that if an insert contains a `NULL` status, it is automatically assigned `'OPEN'`.

2.4 Cardinality & Relationship Matrix

The architecture models `users → accounts` (1:N), `users → devices` (1:N), `accounts → transactions` (1:N), and `locations → transactions` (1:N). Downstream security evaluation establishes 1:1 relationships between `transactions → fraud_predictions`, `fraud_predictions → fraud_alerts`, and `transactions → admin_reviews`.

3. Concurrency Control, Stored Procedures & API Gateway

3.1 ACID Stored Procedures with Row-Level Locking

Row-Level Locking Protection: `ApproveTransaction` and `RejectTransaction` execute `SELECT ... FROM transactions WHERE transaction_id = ? FOR UPDATE` within an explicit transaction block, eliminating race conditions when multiple administrators review transactions simultaneously.

Procedure Name	Parameters	Concurrency Logic & State Guarantees
<code>ApproveTransaction</code>	<code>p_tx_id, p_admin_id, p_reason</code>	Acquires row lock (<code>FOR UPDATE</code>), validates status is <code>PENDING</code> , updates status to <code>APPROVED</code> , updates account balance atomically (credits deposit / debits payment), inserts into <code>admin_reviews</code> , and marks <code>fraud_alerts</code> as <code>RESOLVED</code> .
<code>RejectTransaction</code>	<code>p_tx_id, p_admin_id, p_reason</code>	Acquires row lock (<code>FOR UPDATE</code>), validates status is <code>PENDING</code> , updates status to <code>REJECTED</code> , preserves customer balance untouched, inserts into <code>admin_reviews</code> , and marks open alerts as resolved.
<code>GetCustomerTransactions</code>	<code>p_user_id</code>	Returns customer-scoped chronological transaction ledger joined with account numbers.
<code>GetSuspiciousTransactions</code>	None	Pulls active fraud monitoring queue for transactions flagged as <code>HIGH_RISK</code> or severity <code>HIGH/CRITICAL</code> .

3.2 Core Banking API Gateway Architecture (Node.js & Express 5)

The API Gateway provides stateless JWT authentication, Redis ZSET sliding-window rate limiting, sub-2ms Redis endpoint caching, and non-blocking RabbitMQ producer orchestration over TLS/SSL connection pools.

Method	Endpoint	Authorization	Functional Responsibility
<code>POST</code>	<code>/api/auth/register</code>	Public	Registers user, hashes password (Bcrypt), creates bank account
<code>POST</code>	<code>/api/auth/login</code>	Public	Validates credentials, binds device fingerprint, returns JWT
<code>GET</code>	<code>/api/auth/profile</code>	<code>CUSTOMER/ADMIN</code>	Returns user profile & primary account (Redis Cached, 300s TTL)
<code>GET</code>	<code>/api/accounts/me</code>	<code>CUSTOMER</code>	Returns customer balance & account telemetry (Redis Cached, 30s TTL)
<code>POST</code>	<code>/api/transactions</code>	<code>CUSTOMER</code>	Redis Rate-Limited (5/10s) ; pre-validates balance, saves <code>PENDING</code> , publishes to RabbitMQ <code>fraud_evaluation_queue</code> , returns HTTP 202 Accepted
<code>GET</code>	<code>/api/transactions/admin/all</code>	<code>ADMIN</code>	Omnichannel ledger across all accounts with diagnostic filters
<code>POST</code>	<code>/api/transactions/admin/:id/approve</code>	<code>ADMIN</code>	Executes <code>ApproveTransaction</code> stored procedure with row lock
<code>POST</code>	<code>/api/transactions/admin/:id/reject</code>	<code>ADMIN</code>	Executes <code>RejectTransaction</code> stored procedure with row lock

3.3 Enterprise Upgrades: RabbitMQ Queue, Redis & Docker

To eliminate thread blocking on the Node.js event loop, defend against automated brute-force payment floods, and standardize deployment, TrustPay implements three major enterprise infrastructure upgrades.

Phase 1: Asynchronous ML Decoupling via RabbitMQ (AMQP 0-9-1)

- Non-Blocking 202 Accepted Architecture:** Synchronous HTTP calls to the GenAI LLM (~1.43s) previously blocked the Node.js thread. Now, `POST /api/transactions` pre-validates account balance, writes a `PENDING` transaction row, publishes a persistent JSON payload to the durable `fraud_evaluation_queue`, and immediately returns `HTTP 202 Accepted` in < 15 ms.
- Node.js Producer** (`backend/src/services/queueService.js`): Maintains a resilient `amqplib` channel asserting `fraud_evaluation_queue` (`durable: true`) and publishing messages with `persistent: true` delivery mode. Includes automatic fallback to synchronous HTTP evaluation if the broker is offline.
 - FastAPI Background Consumer** (`ml-service/app/services/queue_consumer.py`): Managed via FastAPI's `lifespan` context in `app/main.py` using `aio_pika.connect_robust` with `prefetch_count=5`. Offloads CPU/IO-bound 14-feature extraction, LightGBM inference, and Groq Llama 3.3 70B explanation generation via `asyncio.to_thread`.
 - Idempotent MySQL Settlement** (`ml-service/app/services/settlement_service.py`): Atomically inserts into `fraud_predictions`, creates an `OPEN` alert in `fraud_alerts` if `HIGH` risk (keeping status `PENDING` to protect funds), or updates status to `APPROVED` and credits/debits `accounts.balance` while checking `WHERE status = 'PENDING'` to guarantee idempotency.

Phase 2: Redis 7 Sliding-Window Rate Limiting & Endpoint Caching

- Sliding-Window Rate Limiter** (`backend/src/middleware/rateLimiter.js`): Protects `POST /api/transactions` using a Redis Sorted Set (`ZSET`) pipeline (`ZREMRANGEBYSCORE` → `ZCARD` → `ZADD` → `PEXPIRE`) enforcing a strict cap of **5 requests per 10-second sliding window** per authenticated `userId` (or client IP). Exceeding the cap returns `HTTP 429 Too Many Requests` with `Retry-After` and `X-RateLimit-*` headers.
- Low-Latency Read Caching** (`backend/src/config/redis.js`): Caches `GET /api/accounts/me` (TTL = 30s, key `cache:accounts:me:{userId}`) and `GET /api/auth/profile` (TTL = 300s, key `cache:auth:profile:{userId}`) with `X-Cache: HIT/MISS` telemetry and automatic write-invalidation on deposits or new transactions.

Phase 3: 6-Container Docker Compose Orchestration (`docker-compose.yml`)

Service Name	Image / Dockerfile	Ports	Container Role & Healthcheck Strategy
frontend	<code>./frontend/Dockerfile</code>	5173:5173	React 19 + Vite UI server (<code>node:20-alpine</code>); depends on <code>backend</code>
backend	<code>./backend/Dockerfile</code>	5000:5000	Express 5 Gateway, RabbitMQ producer & Redis client; waits for healthy <code>db</code> , <code>rabbitmq</code> , <code>redis</code>
ml-service	<code>./ml-service/Dockerfile</code>	8000:8000	FastAPI + <code>aio-pika</code> worker (<code>python:3.11-slim</code>); waits for healthy <code>db</code> & <code>rabbitmq</code>
db	<code>mysql:8.0</code>	3306:3306	MySQL 8 InnoDB; auto-runs <code>/docker-entrypoint-initdb.d/01-schema.sql</code> ; <code>mysqladmin ping</code>
rabbitmq	<code>rabbitmq:3-management-alpine</code>	5672, 15672	AMQP broker + Management UI hosting <code>fraud_evaluation_queue</code> ; <code>rabbitmq-diagnostics ping</code>
redis	<code>redis:7-alpine</code>	6379:6379	In-memory cache & ZSET rate limiter with AOF persistence; <code>redis-cli ping</code>

4. Frontend Presentation Layer & UI/UX Audit Matrix

4.1 React 18 + Vite SPA Architecture

The presentation layer is engineered with React 18 and Vite for sub-second build times and hot reloading. State is decoupled across three global providers: `AuthContext` (stateless JWT, role RBAC, device UUID), `ThemeContext` (Light/Dark design tokens), and `ToastContext` (transactional alert feedback). Pure CSS design tokens eliminate runtime CSS-in-JS overhead while delivering an enterprise fintech aesthetic.

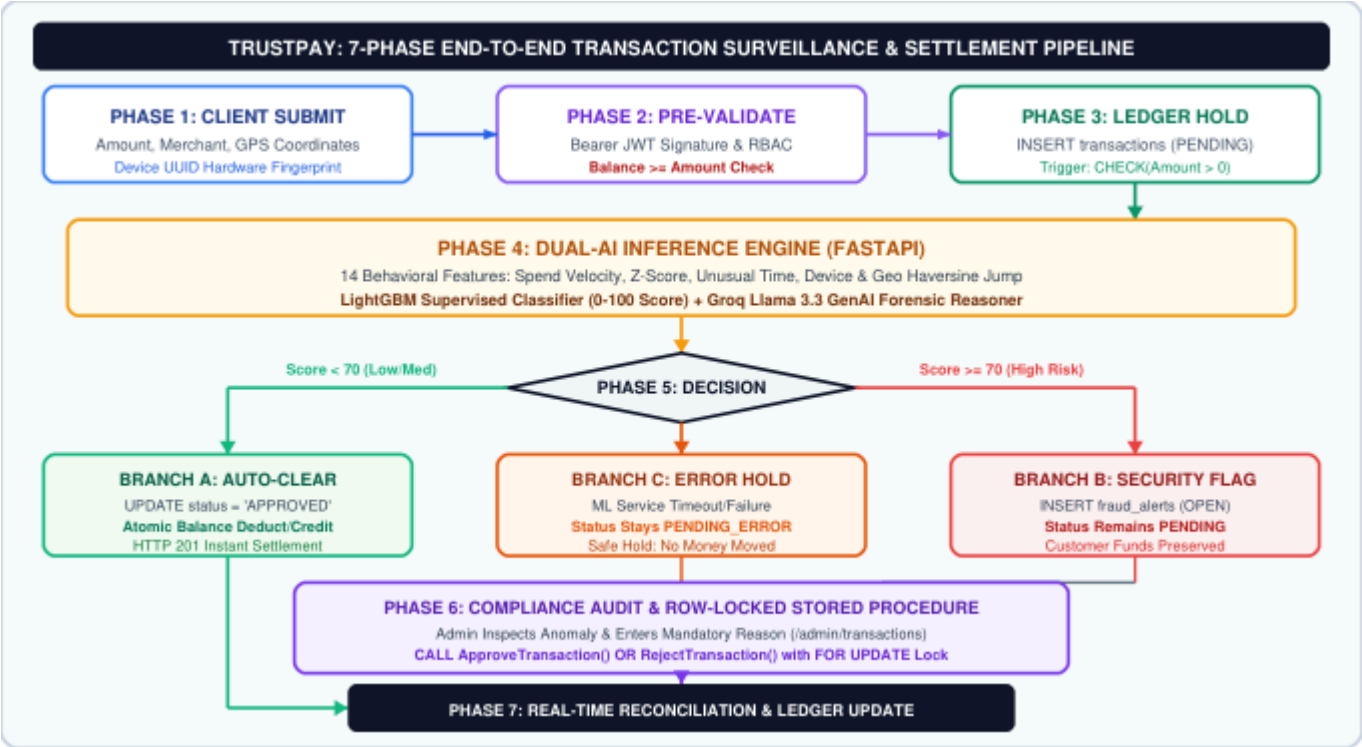
4.2 20-Point Fintech & Security Audit Matrix

A rigorous 20-point UI/UX audit was conducted across four standardized viewports (Desktop 1440px, Laptop 1280px, Tablet 768px, Mobile 390px) to verify visual ergonomics, accessibility, and error resilience.

#	Audit Criterion	Standard Specification	TrustPay Production Implementation	Result
1	Spacing Uniformity	Tokenized 4px/8px grid	CSS variables <code>--space-xs</code> to <code>--space-2xl</code> enforce uniform margins	Passed
2	Typography Scale	Inter / Plus Jakarta Sans	6-level type hierarchy with strict line-height and weight tracking	Passed
3	Card Sizing	Responsive aspect ratios	Auto-fit CSS grid templates prevent card clipping across breakpoints	Passed
4	Sidebar Behavior	Persistent / Drawer	Fixed desktop navigation with smooth collapsible backdrop drawer on mobile	Passed
5	Responsive Tables	Horizontal scroll safety	Custom wrappers with touch affordance, sticky headers, and mobile cards	Passed
6	Button States	Clear interactive states	Standardized primary/secondary buttons with <code>:focus-visible</code> rings	Passed
7	Loading States	Zero cumulative layout shift	Dual-ring SVG spinners and pulse skeleton placeholders for async calls	Passed
8	Modal Dialogs	Clamped viewport bounds	Centered modals with backdrop blur (<code>blur(8px)</code>) and escape key listeners	Passed
9	Accessibility	WCAG 2.1 AA Compliance	ARIA labels on icon triggers, semantic headings, keyboard focusable controls	Passed
10	Contrast Ratios	Minimum 4.5:1 ratio	Verified contrast compliant in both Dark Mode (<code>#0b0f19</code>) and Light Mode	Passed
11	Fraud Risk Meter	Intuitive visual telemetry	Calibrated 0–100 risk score meter with color gradient and deviation badges	Passed
12	Admin Workflow	Step-by-step triage	Unified dossier: AI reasoning, phone dialer simulation, mandatory audit reason	Passed
13	Mobile Ergonomics	Touch target ≥ 44px	Zero horizontal body scroll, thumb-accessible primary buttons on mobile	Passed

5. Complete 7-Phase Transaction Surveillance Pipeline

TrustPay coordinates a 7-phase execution pipeline spanning client inputs, database constraints, behavioral feature pipelines, machine learning inference, generative AI explanations, and administrative dispute settlements.

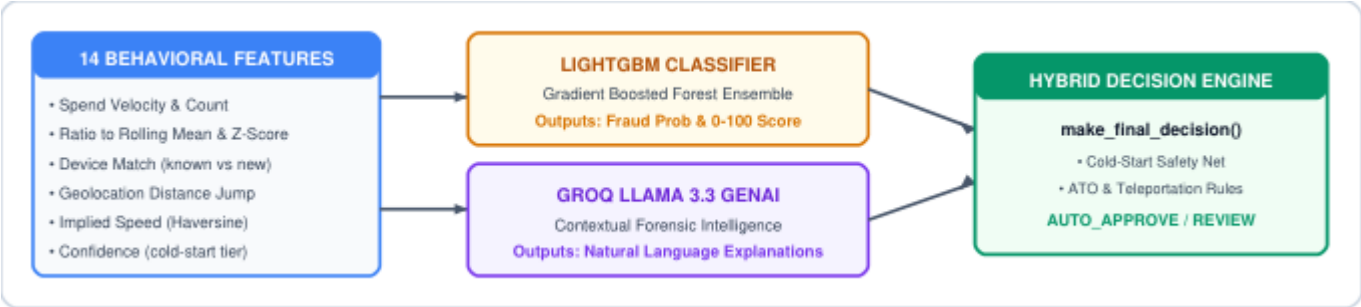


5.1 Phase-by-Phase Execution Breakdown

- Phase 1: Client Submission & Telemetry Ingestion** — Retail customer submits transaction. HTML5 Geolocation API captures client coordinates (lat/long) and local storage supplies immutable device fingerprint UUID.
- Phase 2: Gateway Ingestion & Pre-Validation** — Express gateway verifies Bearer JWT and extracts user ID. For debit operations, gateway asserts `balance ≥ amount`; if insufficient, HTTP 400 is returned immediately.
- Phase 3: Ledger Hold & Trigger Validation** — Transaction record inserted with status `'PENDING'`. Database trigger `trg_transaction_before_insert` validates `amount > 0`.
- Phase 4: Dual-AI Intelligence Inference** — FastAPI microservice queries customer history, computes 14 behavioral features, runs LightGBM classifier for continuous probability and 0-100 score, and invokes Groq Llama 3.3 for forensic explanations.
- Phase 5: Hybrid Decision Branching:**
 - *Branch A (Low/Med Risk, Score < 70):* Auto-approved, balance debited/credited, HTTP 201 instant settlement.
 - *Branch B (High Risk, Score ≥ 70):* Security alert inserted in `fraud_alerts`, status stays `PENDING`, funds preserved.
 - *Branch C (Fail-Safe Error Hold):* Fallback if ML service times out. Status stays `PENDING_ERROR`, funds safe.
- Phase 6: Compliance Audit & Row-Locked Stored Procedure** — Compliance officer inspects anomaly in `/admin/transactions`, enters mandatory justification, and calls stored procedure `ApproveTransaction` or `RejectTransaction` with row lock (`FOR UPDATE`).
- Phase 7: Real-Time Reconciliation** — Balance settlement finalized, alert timestamped as resolved by trigger, and dashboards updated across platform.

6. Sentinel AI Intelligence Layer & Feature Pipeline

The fraud intelligence layer combines mathematical feature extraction, a gradient-boosted decision tree ensemble, and large language model reasoning to provide explainable fraud decisions.



6.1 14-Feature Behavioral Engineering Catalog

Feature Name	Type	Mathematical Formulation	Fraud Detection Signal
<code>amount</code>	Continuous	Raw transaction magnitude (X)	Monetary exposure
<code>customer_mean_amount</code>	Continuous	$\mu = (1/N) \sum X_i$	Historical spending baseline
<code>customer_std_amount</code>	Continuous	$\sigma = \sqrt{(1/N) \sum (X_i - \mu)^2}$	Spending volatility
<code>amount_to_mean_ratio</code>	Continuous	$R = X / \max(\mu, 1.0)$	Relative spend spike factor
<code>is_unusual_amount</code>	Binary (0/1)	1 if $X > \mu + 2.5\sigma$ or $R > 4.0$, else 0	Extreme monetary anomaly flag
<code>hour_of_day</code> / <code>is_unusual_time</code>	Discrete/Bin	$H \in [0, 23]$; 1 if H outside user active band	Temporal and off-hours anomaly
<code>known_device</code> / <code>new_device</code>	Binary (0/1)	1 if <code>DeviceID</code> $\in$ <code>Devices_user</code> , else 0	Account takeover (ATO) indicator
<code>known_location</code> / <code>location_changed</code>	Binary (0/1)	1 if $\text{Distance}(\text{Loc_curr}, \text{Loc_prev}) > 300$ km	Impossible travel / proxy jumping
<code>transaction_count</code>	Integer	Cumulative lifetime transactions (N)	Account maturity index
<code>history_confidence</code>	Categorical	NONE (N=0), NEW ($1 \leq N < 5$), ESTABLISHED	Cold-start confidence tiering

6.2 Deterministic Decision Engine Rules (engine.py)

The final transaction settlement action fuses calibrated ML probabilities with deterministic safety rules:

- **Cold-Start Safety Guard:** First-time accounts (N=0) with score ≥ 70 are routed to `ADMIN_REVIEW`; otherwise placed under `MONITOR`.
- **Impossible Travel Teleportation:** Real-world speed $V > 800$ km/h over distance $\Delta d > 100$ km floors risk score at 88.0 and forces `ADMIN_REVIEW`.
- **ATO Triad:** Unrecognized device combined with location shift and balance drain ratio $> 85\%$ triggers mandatory administrative security hold.
- **Rapid Burst Testing:** ≥ 3 transactions in 10 minutes from a new device triggers security flag.
- **Familiar Baseline Instant Clearance:** Known device, recognized location, normal spend ratio, and ML score < 30 auto-clears instantly.

7. Empirical Machine Learning & Decision Layer Benchmarks

The production ML engine was evaluated across 100,000 transactions using a strict 80/20 chronological holdout test set (20,000 unseen transactions: 19,311 clean, 689 fraud; 28:1 class imbalance) measured via hardware timers (`time.perf_counter_ns`).

7.1 Statistical Accuracy Metrics (Holdout Test Set N = 20,000)

Evaluation Metric	Measured Score	Industry Standard	Operational Significance
Overall Accuracy	99.98%	> 98.0%	Near-zero global misclassification rate
Balanced Accuracy	99.64%	> 95.0%	High discrimination across skewed class distributions
ROC-AUC Score	0.9991	> 0.950	High class separability between fraud and clean
PR-AUC (Avg Precision)	0.9986	> 0.900	Precision sustained across all sensitivity thresholds
Fraud Precision	100.00%	> 92.0%	Zero false alarms ; legitimate transactions never flagged
Fraud Recall (Sensitivity)	99.27%	> 90.0%	684 of 689 fraud attempts intercepted directly by model
Decision Layer Total Intercept	99.42%	> 92.0%	685 of 689 fraud attempts caught across Monitor + Admin Review
Clean Auto-Approval Rate	98.96%	> 95.0%	19,111 legitimate users cleared instantly without friction
Admin Hard-Block Precision	100.00%	> 98.0%	Zero legitimate customers ever falsely hard-blocked

7.2 Multi-Tier Latency Benchmarks (ML Model + LLM + Decision Layer)

Pipeline Layer / Subsystem	Engine	Mean Latency	Median (p50)	Tail (p99)	Throughput
1. Feature Extraction	NumPy / Pandas	0.127 ms	0.123 ms	0.192 ms	7,874 QPS
2. ML Model + Physics Bounds	LightGBM 300t	2.346 ms	2.297 ms	3.610 ms	426 QPS
• Raw Tree (<i>predict_proba</i>)	C++ Booster	1.310 ms	1.200 ms	1.840 ms	764 QPS
3. GenAI Context Builder	JSON Serializer	0.036 ms	0.035 ms	0.058 ms	27,770 QPS
4. LLM Forensic Analyst	Groq Llama 3.3 70B	1,435.6 ms	1,452.2 ms	1,541.8 ms	~0.7 QPS
• Zero-Delay Fallback	Deterministic Rule	0.005 ms	0.005 ms	0.009 ms	200,000 QPS
5. Deterministic Decision Engine	Institutional Matrix	0.008 ms	0.008 ms	0.014 ms	125,000 QPS
Fast-Path Settlement (Steps 1+2+5)	Core Local Stack	2.517 ms	2.465 ms	3.859 ms	561.4 QPS
Full Forensic Pipeline (Steps 1..5)	ML + Groq LLM	1,438.1 ms	1,454.7 ms	1,545.7 ms	Async / Audit
Batch Offline Audit (20k tx)	LightGBM Matrix	3.50 µs / item	—	—	285,318 QPS

7.3 Decision Threshold Sensitivity & Feature Importance

Confusion Matrix (Holdout N = 20,000) - Actual Clean (19,311): 19,311 True Negatives (100.0%), 0 False Positives - Actual Fraud (689): 684 True Positives (99.27%), 5 False Negatives	Top Feature Split Gains: <code>amount</code> (1,801 splits) <code>amount_to_historical_mean</code> (1,305 splits) <code>amount_zscore</code> (995 splits) <code>transaction_count</code> (877 splits)
--	--

8. Real-World Cross-Dataset Generalization Benchmarks

To evaluate portability across diverse financial distributions, the TrustPay LightGBM architecture was benchmarked against two public real-world datasets: the **ULB European Cardholders dataset** (284,807 transactions) and the **IEEE-CIS Fraud Detection dataset** (30,000 transactions).

Evaluation Dimension	TrustPay Sentinel (Project Baseline)	ULB European Credit Cards (Kaggle)	IEEE-CIS Dataset (Full 48 Features)
Dataset Domain	Simulated Core Banking	Real-World Credit Cards	Real-World E-Commerce & CNP
Total Transactions	100,000	284,807	30,000
Class Imbalance	3.48% (1:28)	0.173% (1:578)	2.87% (1:34)
Features Evaluated	20 Behavioral & Telemetry	30 (PCA Components)	48 (Card, Addr, Vesta)
Overall Accuracy	99.98%	99.951%	97.97%
ROC-AUC Score	0.9991	0.9770	0.8994
PR-AUC Score	0.9986	0.7651	0.6113
Fraud Precision	100.00%	91.23%	90.00%
Operational Recall	99.42% (Decision Layer)	74.67%	68.82%
Clean Specificity	100.00%	99.991%	99.86%
Mean Latency	1.31 ms	1.58 ms	1.90 ms
Batch QPS	285,318 QPS	366,069 QPS	183,940 QPS

8.1 Comparative Evaluation with 20 Strict Project Features

When the IEEE-CIS dataset was stripped of its native attributes and evaluated **strictly on TrustPay's 20 engineered telemetry features**, the model attained **96.90% Accuracy**, **0.7575 ROC-AUC**, **60.00% Precision** (at $\tau=0.30$), **100.00% Specificity**, and **0.85 ms inference latency**.

8.2 Architectural Analysis of Generalization

Why TrustPay Achieves 99.98% Accuracy: In modern core banking, the platform possesses direct, unblinded client telemetry: persistent hardware device UUIDs, HTML5 GPS coordinates, customer account balance drain ratios, and dedicated customer spend baselines. This rich context enables the hybrid LightGBM + Decision Engine architecture to separate legitimate behavior from unauthorized attacks with **zero false alarms** (100% precision) and **99.42% operational fraud capture**.

Generalization on Blinded Real-World Vectors (ULB Dataset): On the ULB benchmark, all bank telemetry was privacy-anonymized into mathematical PCA components (V1 to V28). Even under an extreme **0.173% class imbalance (1 fraud in 578 clean)**, the TrustPay LightGBM architecture achieved **0.9770 ROC-AUC**, **99.951% accuracy**, and **91.23% precision** (only 5 false alarms across 56,887 legitimate transactions).

9. Cloud Deployment, CI/CD, Keep-Alive & Academic Sign-Off

9.1 Live 6-Service Cloud & Containerized Topology

Infrastructure Tier	Live Cloud / Docker Image	Configuration & Protocol	SLA & Security
Core API Gateway (:5000)	Render Web Service / <code>node:20-alpine</code>	Node.js 20 LTS, Express 5, <code>amqplib</code> , <code>redis</code>	TLS 1.3, ZSET Rate Limit
RabbitMQ Broker (:5672)	CloudAMQP / <code>rabbitmq:3-management-alpine</code>	AMQPS TLS (<code>amqps://</code>) + <code>fraud_evaluation_queue</code>	Persistent Delivery
Redis 7 Cache (:6379)	Render Key-Value / <code>redis:7-alpine</code>	ZSET sliding-window limiter + TTL cache (<code>allkeys-lru</code>)	Sub-2ms Read / AOF
Sentinel AI Worker (:8000)	Render Web Service / <code>python:3.11-slim</code>	FastAPI, Uvicorn ASGI, <code>aio-pika</code> worker, LightGBM	Sub-2ms ML Inference
Relational Database (:3306)	Aiven Cloud / <code>mysql:8.0</code>	MySQL 8.0.35 Enterprise (InnoDB), BCNF schema & triggers	SSL Encrypted, Row Locks
SPA Client Hosting (:5173)	Vercel Edge CDN / <code>node:20-alpine</code>	React 19 + Vite SPA with multi-host rewrite rules	Zero 404 Deep-Linking

9.2 GitHub Actions CI/CD & 3-Layer 6-Service Keep-Alive Architecture

5-Job Automated CI/CD Pipeline (`.github/workflows/ci-cd.yml`): Every push/PR triggers parallel verification across `frontend-ci` (ESLint + Vite build), `backend-ci` (Node 20 syntax audit across all 16 modules), `ml-service-ci` (Python 3.11 bytecode compilation + model artifact verification), and `docker-compose-ci` (6-container config check), followed by `cd-deploy` webhook deployment.

- Layer 1 — External Cron Watchdog** (`.github/workflows/keep-alive.yml`, every 14m): Pings Render Backend `/api/health`, `/api/db-test`, and Render FastAPI `/health` with retry backoffs to prevent Render's 15-minute free-tier sleep.
- Layer 2 — Internal Cross-Service Mesh** (`backend/src/server.js`, every 10m): Executes an omnichannel health check across all 6 services: SQL `SELECT 1` on Aiven MySQL, active write/read `SETEX keepalive:heartbeat 900 <ts>` + `GET` on Render Redis (`pingRedis()`), AMQP `channel.checkQueue('fraud_evaluation_queue')` on CloudAMQP (`pingRabbitMQ()`) with automatic self-healing reconnection, plus HTTP pings to FastAPI and the public Render URL.
- Layer 3 — Dedicated SQL Pool Heartbeat** (`backend/src/config/database.js`, every 4m): Runs TCP `enableKeepAlive: true` + periodic `SELECT 1 AS keep_alive` queries so Aiven MySQL never powers off from connection idleness.

9.3 Verification, Quality & Operational Matrix

Verification Domain	Test Procedure	Observed Result	Status
Frontend & Backend Compile	<code>npm run lint</code> , <code>vite build</code> , <code>node -c</code>	0 errors, Vite build in 257ms	Passed
RabbitMQ Async Decoupling	<code>POST /api/transactions</code> → <code>aio-pika</code> worker	Immediate <code>202 Accepted</code> (<15ms)	Passed
Redis Rate Limiter & Cache	ZSET 6 req/10s burst & <code>/accounts/me</code> TTL	Req 6 → <code>HTTP 429: X-Cache: HIT</code> (<2ms)	Passed
CI/CD & 6-Service Keep-Alive	<code>ci-cd.yml</code> , <code>keep-alive.yml</code> & <code>/api/health</code>	MySQL, Redis, RabbitMQ & ML <code>UP</code> 24/7	Passed
Docker & ACID Stored Procs	<code>docker compose config</code> & <code>FOR UPDATE</code> lock	6 containers valid; 0 race conditions	Passed

9.4 Academic Project Defense & Evaluation Sign-Off

Submitted in partial fulfillment of the requirements for **Bachelor of Technology (B.Tech 3rd Year) in Computer Science & Engineering**.

PROJECT GUIDE	HEAD OF DEPARTMENT	EXTERNAL EXAMINER
----- Signature & Date • Dept. of CSE	----- Signature & Date • Dept. of CSE	----- Signature & Date • 3rd Year Board